

[ssl] Clés & Certificats – Génération

DESCRIPTION : Génère des clés privées et des certificats X.509

POURQUOI : TLS/HTTPS, authentification, signature

CONTEXTE : Serveurs web, APIs sécurisées, PKI interne

— CLÉS PRIVÉES —

```
openssl genrsa -out cle.pem 2048      Clé RSA 2048 bits
openssl genrsa -out cle.pem 4096      Clé RSA 4096 bits
openssl genrsa -aes256 -out cle.pem 4096 Clé RSA chiffrée (AES-256)
openssl genrsa -aes128 -out cle.pem 2048 Clé RSA chiffrée (AES-128)
openssl genrsa -passout pass:motdepasse -out cle.pem 2048 Mot de passe inline
openssl genrsa -passout file:mdp.txt -out cle.pem 2048 Mot de passe depuis fichier
```

```
openssl genpkey -algorithm RSA -out cle.pem \
  -pkeyopt rsa_keygen_bits:2048      Clé RSA (nouvelle API)
openssl genpkey -algorithm RSA -aes-256-cbc \
  -pass pass:secret -out cle.pem \
  -pkeyopt rsa_keygen_bits:4096      RSA chiffrée (nouvelle API)
```

```
openssl genpkey -algorithm EC \
  -pkeyopt ec_paramgen_curve:P-256 -out ec.pem      Clé EC P-256 (ECDSA/ECDH)
openssl genpkey -algorithm EC \
  -pkeyopt ec_paramgen_curve:P-384 -out ec.pem      Clé EC P-384
openssl genpkey -algorithm EC \
  -pkeyopt ec_paramgen_curve:P-521 -out ec.pem      Clé EC P-521
openssl genpkey -algorithm EC \
  -pkeyopt ec_paramgen_curve:secp256k1 -out ec.pem Clé EC secp256k1 (Bitcoin)
openssl genpkey -algorithm X25519 -out x25519.pem Clé X25519 (DH Curve25519)
openssl genpkey -algorithm Ed25519 -out ed25519.pem Clé Ed25519 (signature)
openssl genpkey -algorithm Ed448 -out ed448.pem Clé Ed448 (signature)
openssl genpkey -algorithm X448 -out x448.pem Clé X448 (DH)
```

```
openssl ecparam -name prime256v1 -genkey -out ec.pem Clé EC P-256 (ancienne API)
openssl ecparam -list_curves      Liste toutes les courbes EC
```

```
openssl dhparam -out dh.pem 2048      Paramètres Diffie-Hellman
openssl dhparam -out dh.pem 4096      DH 4096 bits (lent)
```

— EXPORT / CONVERSION DE CLÉS —

```
openssl rsa -in cle.pem -out cle_pub.pem -pubout      Extraire clé publique RSA
openssl pkey -in cle.pem -out cle_pub.pem -pubout      Extraire clé publique (generic)
openssl rsa -in cle.pem -out cle_no_pass.pem          Supprimer le mot de passe
openssl rsa -in cle.pem -aes256 -out cle_enc.pem      Ajouter un mot de passe
openssl rsa -in cle.pem -text -noout                  Afficher les détails RSA
openssl pkey -in cle.pem -text -noout                  Afficher détails (generic)
openssl rsa -in cle.pem -check                        Vérifier l'intégrité RSA
```

Formats

```
openssl rsa -in cle.pem -outform DER -out cle.der      PEM → DER (binaire)
openssl rsa -inform DER -in cle.der -out cle.pem      DER → PEM
openssl pkcs8 -topk8 -in cle.pem -out cle_p8.pem      RSA → PKCS#8 PEM
openssl pkcs8 -topk8 -nocrypt -in cle.pem -out cle_p8.pem PKCS#8 sans chiffrement
openssl pkcs8 -topk8 -in cle.pem -outform DER -out cle.p8 PKCS#8 DER
```

— CSR (Certificate Signing Request) —

```
openssl req -new -key cle.pem -out req.csr      CSR depuis clé existante
openssl req -newkey rsa:2048 -nodes -keyout cle.pem \
  -out req.csr      Clé + CSR en une commande
openssl req -newkey rsa:2048 -keyout cle.pem \
  -out req.csr -subj "/CN=example.com/O=MonOrg/C=FR"      CSR avec sujet inline
openssl req -new -key cle.pem -out req.csr \
  -subj "/C=FR/ST=IDF/L=Paris/O=Org/OU=IT/CN=host.com"      CSR détaillé
openssl req -in req.csr -text -noout            Lire un CSR
openssl req -in req.csr -verify                  Vérifier un CSR
openssl req -in req.csr -noout -subject          Afficher le sujet
```

— CERTIFICATS AUTO-SIGNÉS —

```
openssl req -x509 -newkey rsa:2048 -nodes \
```

-keyout cle.pem -out cert.pem -days 365	Cert auto-signé 1 an
openssl req -x509 -newkey rsa:4096 \	
-keyout cle.pem -out cert.pem -days 3650 \	
-subj "/CN=monCA/O=MonOrg/C=FR"	CA auto-signé 10 ans
openssl x509 -req -in req.csr -signkey cle.pem \	
-out cert.pem -days 365	Signer un CSR
openssl x509 -req -in req.csr -CA ca.pem -CAkey ca.key \	
-CAcreateserial -out cert.pem -days 365	Signer avec une CA
openssl x509 -req -in req.csr -CA ca.pem -CAkey ca.key \	
-CAcreateserial -out cert.pem -days 365 \	
-extfile ext.cnf -extensions v3_req	Avec extensions X.509 v3

— LECTURE / INSPECTION DE CERTIFICATS —

openssl x509 -in cert.pem -text -noout	Détails complets
openssl x509 -in cert.pem -noout -subject	Sujet du certificat
openssl x509 -in cert.pem -noout -issuer	Émetteur
openssl x509 -in cert.pem -noout -dates	Dates de validité
openssl x509 -in cert.pem -noout -serial	Numéro de série
openssl x509 -in cert.pem -noout -fingerprint	Empreinte SHA-1
openssl x509 -in cert.pem -noout -fingerprint -sha256	Empreinte SHA-256
openssl x509 -in cert.pem -noout -pubkey	Clé publique du cert
openssl x509 -in cert.pem -noout -modulus md5sum	Modulus RSA
openssl x509 -inform DER -in cert.der -text -noout	Depuis format DER
openssl x509 -in cert.pem -outform DER -out cert.der	PEM → DER
openssl x509 -inform DER -in cert.der -out cert.pem	DER → PEM

— PKCS#12 (.pfx / .p12) —

openssl pkcs12 -export -in cert.pem -inkey cle.pem \	
-out bundle.p12 -name "alias"	Créer un .p12
openssl pkcs12 -export -in cert.pem -inkey cle.pem \	
-certfile ca.pem -out bundle.p12	Avec chaîne CA
openssl pkcs12 -in bundle.p12 -out tout.pem -nodes	Extraire tout (PEM)
openssl pkcs12 -in bundle.p12 -nokeys -out cert.pem	Extraire le certificat
openssl pkcs12 -in bundle.p12 -nocerts -nodes \	
-out cle.pem	Extraire la clé privée
openssl pkcs12 -in bundle.p12 -info	Afficher le contenu

— CHAÎNES DE CERTIFICATS —

cat cert.pem intermediate.pem ca.pem > chain.pem	Concaténer la chaîne
openssl verify -CAfile ca.pem cert.pem	Vérifier avec CA
openssl verify -CAfile ca.pem -untrusted inter.pem \	
cert.pem	Vérifier chaîne complète
openssl x509 -in cert.pem -noout -issuer_hash	Hash émetteur
openssl x509 -in cert.pem -noout -subject_hash	Hash sujet

FICHER ext.cnf POUR EXTENSIONS SAN :

```
[req]
distinguished_name = req_distinguished_name
req_extensions = v3_req
[req_distinguished_name]
[v3_req]
subjectAltName = @alt_names
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
[alt_names]
DNS.1 = example.com
DNS.2 = www.example.com
IP.1 = 192.168.1.10
```

[ssl] Chiffrement Symétrique – enc

DESCRIPTION : Chiffre et déchiffre des fichiers avec clés symétriques
 POURQUOI : Protéger des fichiers sensibles, sauvegardes chiffrées
 CONTEXTE : Archivage sécurisé, transmission de fichiers

CHIFFREMENT :

openssl enc -aes-256-cbc -in clair.txt -out chiffré.bin \	
-pass pass:secret -pbkdf2	Chiffrer AES-256-CBC
openssl enc -aes-256-gcm -in clair.txt -out chiffré.bin \	
-pass pass:secret -pbkdf2	AES-256-GCM (authentifié)
openssl enc -aes-256-cbc -in clair.txt -out chiffré.bin \	
-pass file:mdp.txt -pbkdf2 -iter 100000	Mot de passe fichier
openssl enc -aes-256-cbc -in clair.txt -out chiffré.b64 \	

```

    -pass pass:secret -pbkdf2 -base64
openssl enc -aes-256-cbc -nosalt -K hexkey -iv hexiv \
    -in clair.txt -out chiffré.bin

```

Sortie en base64
Clé et IV en hex direct

DÉCHIFFREMENT :

```

openssl enc -aes-256-cbc -d -in chiffré.bin -out clair.txt \
    -pass pass:secret -pbkdf2
openssl enc -aes-256-cbc -d -in chiffré.b64 -out clair.txt \
    -pass pass:secret -pbkdf2 -base64

```

Déchiffrer
Depuis base64

ALGORITHMES DISPONIBLES :

```

openssl enc -list
-aes-128-cbc, -aes-192-cbc, -aes-256-cbc
-aes-128-gcm, -aes-256-gcm
-aes-128-cfb, -aes-256-cfb
-aes-128-ofb, -aes-256-ofb
-aes-128-ctr, -aes-256-ctr
-camellia-256-cbc
-chacha20
-chacha20-poly1305
-des3
-rc4

```

Liste tous les algorithmes
AES CBC
AES GCM (AEAD)
AES CFB
AES OFB
AES CTR
Camellia
ChaCha20
ChaCha20-Poly1305 (AEAD)
3DES (déprécié)
RC4 (ne pas utiliser)

DÉRIVATION DE CLÉ :

```

# -pbkdf2 : PBKDF2 (recommandé, remplace l'ancien EVP_BytesToKey)
# -iter N : nombre d'itérations PBKDF2 (défaut 10000, recommandé 100000+)
# -md sha256 : algorithme de hash pour la dérivation
openssl enc -aes-256-cbc -in f.txt -out f.enc \
    -pass pass:secret -pbkdf2 -iter 600000 -md sha256

```

Très sécurisé

[ssl] Hashage & HMAC – dgst

DESCRIPTION : Calcule des empreintes cryptographiques et des HMAC
POURQUOI : Vérification d'intégrité, authentification de messages
CONTEXTE : Checksums, signatures de fichiers, tokens

HASHAGE :

```

openssl dgst -sha256 fichier.txt
openssl dgst -sha512 fichier.txt
openssl dgst -sha3-256 fichier.txt
openssl dgst -sha3-512 fichier.txt
openssl dgst -blake2b512 fichier.txt
openssl dgst -blake2s256 fichier.txt
openssl dgst -md5 fichier.txt
openssl dgst -sha1 fichier.txt
openssl dgst -sha256 -hex fichier.txt
openssl dgst -sha256 -binary fichier.txt | xxd
openssl dgst -sha256 -c fichier.txt
cat fichier | openssl dgst -sha256
openssl list -digest-algorithms

```

Hash SHA-256
Hash SHA-512
SHA3-256
SHA3-512
BLAKE2b-512
BLAKE2s-256
MD5 (ne pas utiliser)
SHA-1 (déprécié)
Sortie hexadécimale
Sortie binaire
Avec deux-points séparateurs
Depuis stdin
Liste des algos

HMAC :

```

openssl dgst -sha256 -hmac "cle_secrete" fichier.txt
openssl dgst -sha512 -hmac "cle_secrete" fichier.txt
echo -n "message" | openssl dgst -sha256 -hmac "cle"
openssl dgst -sha256 -mac HMAC \
    -macopt key:cle_hex fichier.txt

```

HMAC-SHA256
HMAC-SHA512
HMAC depuis stdin
HMAC avec clé hex

SIGNATURE RSA :

```

openssl dgst -sha256 -sign cle.pem -out sig.bin f.txt
openssl dgst -sha256 -verify cle_pub.pem \
    -signature sig.bin f.txt
openssl dgst -sha256 -prverify cle.pem \
    -signature sig.bin f.txt

```

Signer avec clé privée
Vérifier avec clé publique
Vérifier avec clé privée

SIGNATURE AVEC PKEYUTL :

```

openssl pkeyutl -sign -in hash.bin -inkey cle.pem \
    -out sig.bin -pkeyopt digest:sha256
openssl pkeyutl -verify -in hash.bin -sigfile sig.bin \
    -pubin -inkey cle_pub.pem \
    -pkeyopt digest:sha256

```

Signer un hash
Vérifier

[ssl] Chiffrement Asymétrique – rsautl / pkeyutl

DESCRIPTION : Chiffre/déchiffre avec des clés publiques/privées RSA
POURQUOI : Échange de clés, enveloppe numérique, chiffrement de petits messages
CONTEXTE : Échange sécurisé de secrets, wrapping de clés AES

CHIFFREMENT RSA :

```
openssl pkeyutl -encrypt -in clair.bin \
  -pubin -inkey cle_pub.pem -out chiffré.bin      Chiffrer avec clé publique
openssl pkeyutl -decrypt -in chiffré.bin \
  -inkey cle.pem -out clair.bin                    Déchiffrer avec clé privée
openssl pkeyutl -encrypt -in clair.bin \
  -pubin -inkey cle_pub.pem \
  -pkeyopt rsa_padding_mode:oaep \
  -pkeyopt rsa_oaep_md:sha256 \
  -out chiffré.bin                                RSA-OAEP-SHA256 (recommandé)
```

RSAUTL (ancienne API) :

```
openssl rsautl -encrypt -pubin -inkey cle_pub.pem \
  -in clair.bin -out chiffré.bin                    PKCS#1 v1.5 (à éviter)
openssl rsautl -decrypt -inkey cle.pem \
  -in chiffré.bin -out clair.bin
openssl rsautl -sign -inkey cle.pem -in hash.bin \
  -out sig.bin                                       Signer (ancienne API)
openssl rsautl -verify -pubin -inkey cle_pub.pem \
  -in sig.bin -out hash.bin                         Vérifier (ancienne API)
```

[ssl] TLS / Tests de Connexion – s_client, s_server

DESCRIPTION : Teste les connexions TLS et inspecte les certificats de serveurs
POURQUOI : Diagnostic, débogage TLS, inspection des chaînes de certificats
CONTEXTE : DevOps, sécurité, vérification de configurations

S_CLIENT :

```
openssl s_client -connect example.com:443           Connexion TLS
openssl s_client -connect example.com:443 -tls1_3   Forcer TLS 1.3
openssl s_client -connect example.com:443 -tls1_2   Forcer TLS 1.2
openssl s_client -connect example.com:443 \
  -servername example.com                           SNI (Server Name Indication)
openssl s_client -connect example.com:443 \
  -showcerts                                         Afficher toute la chaîne
openssl s_client -connect example.com:443 \
  -CAfile ca.pem                                    Vérifier avec CA custom
openssl s_client -connect example.com:443 \
  -cert cert.pem -key cle.pem                       Authentification client
openssl s_client -connect example.com:443 \
  -state -debug                                       Mode debug détaillé
echo "" | openssl s_client -connect ex.com:443 \
  -servername ex.com 2>/dev/null | openssl x509 \
  -noout -dates                                       Dates du certificat serveur
echo "" | openssl s_client -connect ex.com:443 2>/dev/null \
  | openssl x509 -noout -text                         Détails cert serveur
openssl s_client -connect localhost:25 -starttls smtp STARTTLS SMTP
openssl s_client -connect localhost:143 -starttls imap STARTTLS IMAP
openssl s_client -connect localhost:21 -starttls ftp  STARTTLS FTP
openssl s_client -connect localhost:5432 -starttls postgres STARTTLS PostgreSQL
```

S_SERVER :

```
openssl s_server -cert cert.pem -key cle.pem \
  -port 4433 -www                                     Serveur HTTPS test
openssl s_server -cert cert.pem -key cle.pem \
  -port 4433 -accept 4433 -tls1_3                   Serveur TLS 1.3
openssl s_server -cert cert.pem -key cle.pem \
  -CAfile ca.pem -Verify 1 -port 4433                Authentification client requise
```

INFORMATIONS TLS :

```
openssl ciphers -v 'HIGH:!aNULL:!MD5'              Lister les suites de chiffrement
openssl ciphers -v 'TLSv1.3'                        Suites TLS 1.3
openssl list -tls1-3-cipher-suites                  Suites TLS 1.3 (v3.x)
```

[ssl] Génération de données aléatoires – rand

DESCRIPTION : Génère des octets aléatoires cryptographiquement sûrs
POURQUOI : Clés, IV, nonces, tokens, mots de passe
CONTEXTE : Bootstrap de secrets, génération de tokens

openssl rand 32	32 octets aléatoires (binaire)
openssl rand -hex 32	32 octets en hexadécimal (64 chars)
openssl rand -base64 32	32 octets en base64
openssl rand -hex 16	Clé 128 bits (AES-128)
openssl rand -hex 32	Clé 256 bits (AES-256)
openssl rand -hex 16	IV 128 bits
openssl rand -base64 48 tr -d '\n'	Token URL-safe (64 chars)
openssl rand -base64 64 tr -d '\n/+'	Token alphanuma

[ssl] OCSP, CRL & Révocation

openssl ocsf -issuer ca.pem -cert cert.pem \	
-url http://ocsp.example.com -resp_text	Vérifier OCSP
openssl crl -in crl.pem -text -noout	Lire une CRL
openssl crl -in crl.pem -CAfile ca.pem -verify	Vérifier une CRL
openssl ca -gencrl -out crl.pem -config openssl.cnf	Générer une CRL

[ssl] Encodage – base64, asn1parse

openssl base64 -e -in fichier.bin -out fichier.b64	Encoder en base64
openssl base64 -d -in fichier.b64 -out fichier.bin	Décoder base64
cat fichier.bin openssl base64	Encoder stdin
openssl asn1parse -in cert.pem	Parser ASN.1 PEM
openssl asn1parse -in cert.der -inform DER	Parser ASN.1 DER
openssl asn1parse -in cert.pem -strparse OFFSET	Parser à un offset

[ssl] Benchmarks & Informations

openssl speed	Benchmark tous les algos
openssl speed aes-256-cbc	Benchmark AES-256-CBC
openssl speed sha256	Benchmark SHA-256
openssl speed rsa2048	Benchmark RSA 2048
openssl speed ecdf	Benchmark ECDH
openssl version	Version d'OpenSSL
openssl version -a	Infos complètes
openssl list -public-key-algorithms	Algos clés publiques
openssl list -cipher-algorithms	Algos de chiffrement
openssl list -digest-algorithms	Algos de hash
openssl list -mac-algorithms	Algos MAC
openssl list -kdf-algorithms	Algos de dérivation

[py] Module secrets – Génération sécurisée en Python

DESCRIPTION : Génère des nombres, tokens et secrets cryptographiquement sûrs
POURQUOI : Remplace random pour tout usage lié à la sécurité
CONTEXTE : Tokens d'authentification, mots de passe, clés, nonces

IMPORT :
import secrets

OCTETS ALÉATOIRES :

secrets.token_bytes(32)	32 octets aléatoires
secrets.token_bytes(16)	16 octets (128 bits)
secrets.token_bytes()	Taille par défaut (32 octets)

TOKENS TEXTE :

secrets.token_hex(32)	64 caractères hex
secrets.token_hex(16)	32 caractères hex
secrets.token_hex()	Hex taille par défaut
secrets.token_urlsafe(32)	Base64 URL-safe ~43 chars
secrets.token_urlsafe(16)	Base64 URL-safe ~22 chars

```

secrets.token_urlsafe()                                URL-safe taille par défaut

ENTIERS & CHOIX :
secrets.randbelow(n)                                    Entier aléatoire dans [0, n)
secrets.randbits(k)                                    Entier aléatoire de k bits
secrets.choice(séquence)                               Éléments aléatoires d'une séq.

GÉNÉRATION DE MOTS DE PASSE :
import secrets, string
alphabet = string.ascii_letters + string.digits + string.punctuation
mdp = ''.join(secrets.choice(alphabet) for _ in range(20))  MDP 20 caractères

# Garantir complexité
while True:
    mdp = ''.join(secrets.choice(alphabet) for _ in range(20))
    if (any(c.islower() for c in mdp) and
        any(c.isupper() for c in mdp) and
        any(c.isdigit() for c in mdp) and
        any(c in string.punctuation for c in mdp)):
        break

# Mot de passe lisible (Diceware-like)
mots = ["cheval", "batterie", "correct", "agrafe", "soleil"]
passphrase = '-'.join(secrets.choice(mots) for _ in range(5))

COMPARAISON SÉCURISÉE (résistante aux timing attacks) :
secrets.compare_digest(a, b)                            Comparaison en temps constant
secrets.compare_digest(token_reçu, token_attendu)        Vérification sécurisée

EXEMPLES :
# Token d'authentification API
api_token = secrets.token_urlsafe(32)                    # 43 caractères

# Token de réinitialisation de mot de passe
reset_token = secrets.token_hex(32)                      # 64 caractères hex

# Nonce pour CSRF
nonce = secrets.token_urlsafe(16)

# Sel pour hachage de mot de passe (préférer bcrypt/argon2 en pratique)
sel = secrets.token_bytes(32)

# Vérification sans timing attack
if secrets.compare_digest(token_reçu.encode(), token_stocké.encode()):
    accorder_accès()

```

[py] Module ssl – Connexions TLS en Python

DESCRIPTION : Crée des connexions TLS sécurisées en Python
POURQUOI : HTTPS client/serveur, sockets chiffrés, authentification mutuelle
CONTEXTE : APIs, serveurs custom, proxies, microservices

CONTEXTE SSL :
 import ssl

```

ssl.create_default_context()                            Contexte client par défaut
ssl.create_default_context(ssl.Purpose.SERVER_AUTH)      Client vérifie le serveur
ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)      Serveur vérifie le client
ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)                Client TLS
ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)                 Serveur TLS

```

CONFIGURATION DU CONTEXTE :

```

ctx = ssl.create_default_context()
ctx.load_verify_locations("ca.pem")                    CA de confiance
ctx.load_cert_chain("cert.pem", "cle.pem")             Certificat + clé
ctx.load_cert_chain("cert.pem", "cle.pem", "motdepasse") Avec mot de passe
ctx.verify_mode = ssl.CERT_REQUIRED                   Vérification requise
ctx.verify_mode = ssl.CERT_OPTIONAL                   Vérification optionnelle
ctx.verify_mode = ssl.CERT_NONE                       Pas de vérification
ctx.check_hostname = True                             Vérifie le hostname
ctx.check_hostname = False                            Désactive (déconseillé)
ctx.minimum_version = ssl.TLSVersion.TLSv1_2          Version minimale
ctx.minimum_version = ssl.TLSVersion.TLSv1_3          TLS 1.3 minimum

```

ctx.maximum_version = ssl.TLSVersion.TLSv1_3	Version maximale
ctx.set_ciphers("HIGH:!aNULL:!MD5")	Suites de chiffrement
ctx.set_ciphers("ECDHE+AESGCM:ECDHE+CHACHA20")	Suites modernes
ctx.set_alpn_protocols(["h2", "http/1.1"])	ALPN (HTTP/2)
ctx.set_npn_protocols(["http/1.1"])	NPN (déprécié)
ctx.options = ssl.OP_NO_SSLv2	Désactiver SSLv2
ctx.options = ssl.OP_NO_SSLv3	Désactiver SSLv3
ctx.options = ssl.OP_NO_TLSv1	Désactiver TLS 1.0
ctx.options = ssl.OP_NO_TLSv1_1	Désactiver TLS 1.1
ctx.options = ssl.OP_SINGLE_DH_USE	Ephemeral DH
ctx.options = ssl.OP_SINGLE_ECDH_USE	Ephemeral ECDH

CONNEXION CLIENT :

```
import socket, ssl
ctx = ssl.create_default_context()
with socket.create_connection(("example.com", 443)) as sock:
    with ctx.wrap_socket(sock, server_hostname="example.com") as ssock:
        ssock.send(b"GET / HTTP/1.1\r\nHost: example.com\r\n\r\n")
        data = ssock.recv(4096)
```

```
# HTTPS avec urllib
import urllib.request
ctx = ssl.create_default_context()
response = urllib.request.urlopen("https://example.com", context=ctx)
```

SERVEUR TLS :

```
import socket, ssl, threading
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.load_cert_chain("cert.pem", "cle.pem")
with socket.socket() as srv:
    srv.bind(("0.0.0.0", 443))
    srv.listen(5)
    with ctx.wrap_socket(srv, server_side=True) as ssrv:
        conn, addr = ssrv.accept()
        data = conn.recv(1024)
        conn.send(b"Bonjour!")
```

INFORMATIONS SUR LA CONNEXION :

ssock.version()	Version TLS utilisée
ssock.cipher()	(nom, proto, bits)
ssock.getpeercert()	Certificat du pair
ssock.getpeercert(binary_form=True)	Certificat binaire DER
ssock.selected_alpn_protocol()	Protocole ALPN choisi
ssock.selected_npn_protocol()	Protocole NPN choisi
ssock.shared_ciphers()	Suites communes
ssock.compression()	Méthode de compression
ssl.match_hostname(cert, hostname)	Vérifie le hostname

UTILITAIRES :

ssl.get_default_verify_paths()	Chemins de CAs par défaut
ssl.where()	Chemin openssl.cnf
ssl.OPENSSL_VERSION	Version OpenSSL
ssl.OPENSSL_VERSION_INFO	Tuple version
ssl.HAS_TLSv1_3	True si TLS 1.3 supporté
ssl.HAS_ALPN	True si ALPN supporté
ssl.HAS_SNI	True si SNI supporté
ssl.Purpose.SERVER_AUTH	Constante d'usage
ssl.Purpose.CLIENT_AUTH	Constante d'usage
ssl.TLSVersion.TLSv1_2	Constante version
ssl.TLSVersion.TLSv1_3	Constante version

DÉSACTIVER LA VÉRIFICATION (DEV UNIQUEMENT) :

ctx = ssl._create_unverified_context()	JAMAIS en production
ctx.check_hostname = False	
ctx.verify_mode = ssl.CERT_NONE	

[py] Module hashlib – Hashage & KDF

DESCRIPTION : Calcule des empreintes cryptographiques et dérive des clés
 POURQUOI : Intégrité, stockage de mots de passe, dérivation de clés
 CONTEXTE : Authentification, vérification de fichiers, clés de chiffrement

ALGORITHMES DISPONIBLES :

```

import hashlib
hashlib.algorithms_available
hashlib.algorithms_guaranteed

```

Tous les algos disponibles
Algos garantis sur toutes plateformes

HASH SIMPLE :

```

hashlib.md5(b"data").hexdigest()
hashlib.shal(b"data").hexdigest()
hashlib.sha256(b"data").hexdigest()
hashlib.sha512(b"data").hexdigest()
hashlib.sha3_256(b"data").hexdigest()
hashlib.sha3_512(b"data").hexdigest()
hashlib.blake2b(b"data").hexdigest()
hashlib.blake2s(b"data").hexdigest()

```

MD5 (ne pas utiliser pour sécurité)
SHA-1 (déprécié)
SHA-256
SHA-512
SHA3-256
SHA3-512
BLAKE2b
BLAKE2s

```

h = hashlib.sha256()
h.update(b"partie1")
h.update(b"partie2")
h.hexdigest()

```

Hash cumulatif

```

hashlib.sha256(b"data").digest()
hashlib.sha256(b"data").hexdigest()
hashlib.sha256(b"data").digest_size
hashlib.sha256(b"data").block_size
hashlib.sha256(b"data").name
hashlib.sha256(b"data").copy()

```

Bytes bruts
Chaîne hexadécimale
Taille en octets
Taille du bloc
Nom de l'algorithme
Copie de l'état

```

# Hash d'un fichier
h = hashlib.sha256()
with open("fichier.bin", "rb") as f:
    for bloc in iter(lambda: f.read(65536), b''):
        h.update(bloc)
print(h.hexdigest())

```

BLAKE2 AVEC CLÉ (MAC) :

```

hashlib.blake2b(b"data", key=b"cle_secrete").hexdigest()
hashlib.blake2s(b"data", key=b"cle_16octets").hexdigest()
hashlib.blake2b(b"data", digest_size=32).hexdigest()
hashlib.blake2b(b"data", person=b"moncontexte").hexdigest()

```

BLAKE2b MAC
BLAKE2s MAC
BLAKE2b tronqué
Personnalisation

PBKDF2 (dérivation de clé depuis mot de passe) :

```

hashlib.pbkdf2_hmac(
    hash_name="sha256",
    password=b"motdepasse",
    salt=b"sel_aleatoire",
    iterations=600000,
)
hashlib.pbkdf2_hmac("sha256", b"mdp", sel, 600000, dklen=64)

```

NIST recommande 600k+ (2023)
Retourne 32 octets
Longueur custom

SCRYPT (dérivation de clé résistante GPU) :

```

hashlib.scrypt(
    password=b"motdepasse",
    salt=b"sel_aleatoire",
    n=2**14,
    r=8,
    p=1,
    dklen=32,
)

```

Coût CPU/mémoire (puissance de 2)
Taille de bloc
Parallélisme
Longueur clé dérivée

Recommandations : n=2**20 (1M) pour haut de gamme, n=2**14 pour interactif

HMAC :

```

import hmac
hmac.new(b"cle", b"message", "sha256").hexdigest()
hmac.new(b"cle", b"message", "sha512").digest()
hmac.new(b"cle", digestmod=hashlib.sha256) \
    .update(b"message")
hmac.compare_digest(mac1, mac2)

```

HMAC-SHA256
HMAC-SHA512 bytes
HMAC incrémental
Comparaison sécurisée (timing-safe)

EXEMPLES :

```

import hashlib, secrets

# Hacher un mot de passe avec sel (utiliser bcrypt/argon2 en production)
sel = secrets.token_bytes(32)
cle = hashlib.pbkdf2_hmac("sha256", b"motdepasse", sel, 600000)
# Stocker : sel.hex() + ":" + cle.hex()

```



```
# Vérifier un mot de passe
sel_stocké = bytes.fromhex(sel_hex)
cle_attendue = bytes.fromhex(cle_hex)
cle_calculée = hashlib.pbkdf2_hmac("sha256", b"tentative", sel_stocké, 600000)
if hmac.compare_digest(cle_calculée, cle_attendue):
    print("Mot de passe correct")

# Hash d'intégrité de fichier
def hash_fichier(chemin):
    h = hashlib.sha256()
    with open(chemin, "rb") as f:
        for chunk in iter(lambda: f.read(65536), b''):
            h.update(chunk)
    return h.hexdigest()
```

```
[py] Module cryptography – Chiffrement complet
```

```
DESCRIPTION : Bibliothèque de cryptographie de haut et bas niveau
POURQUOI : Chiffrement symétrique/asymétrique, signatures, certificats X.509
CONTEXTE : Applications nécessitant une crypto robuste et moderne
INSTALLATION : pip install cryptography
```

— CHIFFREMENT SYMÉTRIQUE AES-GCM —

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

cle = AESGCM.generate_key(bit_length=256)          Générer une clé 256 bits
cle = AESGCM.generate_key(128)                    Clé 128 bits
aesgcm = AESGCM(cle)

nonce = secrets.token_bytes(12)                   IV 96 bits (12 octets) REQUIS
chiffré = aesgcm.encrypt(nonce, b"message", b"aad") Chiffrer avec AAD
chiffré = aesgcm.encrypt(nonce, b"message", None) Sans AAD
clair = aesgcm.decrypt(nonce, chiffré, b"aad")     Déchiffrer
clair = aesgcm.decrypt(nonce, chiffré, None)       Sans AAD
# InvalidTag levée si le tag d'authentification est invalide
```

CHACHA20-POLY1305 :

```
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305

cle = ChaCha20Poly1305.generate_key()              Clé 256 bits
chachapoly = ChaCha20Poly1305(cle)
nonce = secrets.token_bytes(12)                    96 bits requis
chiffré = chachapoly.encrypt(nonce, b"message", None)
clair = chachapoly.decrypt(nonce, chiffré, None)
```

AESGCM 128 bits :

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM, AESCCM
cle = AESCCM.generate_key(256)                     CCM mode
aesccm = AESCCM(cle, tag_length=16)
nonce = secrets.token_bytes(13)
chiffré = aesccm.encrypt(nonce, b"message", None)
```

AES-CBC (MANUEL) :

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding

cle = secrets.token_bytes(32)                       32 octets = AES-256
iv = secrets.token_bytes(16)                       16 octets IV
cipher = Cipher(algorithms.AES(cle), modes.CBC(iv))
encryptor = cipher.encryptor()
padder = padding.PKCS7(128).padder()                Padding 128 bits (1 bloc AES)
padded = padder.update(b"message") + padder.finalize()
chiffré = encryptor.update(padded) + encryptor.finalize()

decryptor = cipher.decryptor()
unpadder = padding.PKCS7(128).unpadder()
padded_clair = decryptor.update(chiffré) + decryptor.finalize()
clair = unpadder.update(padded_clair) + unpadder.finalize()
```

AES-CTR :

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
cle = secrets.token_bytes(32)
nonce = secrets.token_bytes(16)
```

```

cipher = Cipher(algorithms.AES(cle), modes.CTR(nonce))
enc = cipher.encryptor()
chiffré = enc.update(b"message") + enc.finalize()

```

— CHIFFREMENT ASYMÉTRIQUE RSA

```

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives import serialization

```

Génération

```

cle_privée = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
)
cle_publique = cle_privée.public_key()

```

Chiffrement OAEP (recommandé)

```

chiffré = cle_publique.encrypt(
    b"message",
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    )
)
clair = cle_privée.decrypt(chiffré,
    padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(), label=None))

```

Signature PSS (recommandée)

```

signature = cle_privée.sign(
    b"message",
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH,
    ),
    hashes.SHA256(),
)

```

Vérification (lève InvalidSignature si invalide)

```

cle_publique.verify(
    signature, b"message",
    padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH),
    hashes.SHA256(),
)

```

Signature PKCS1v15

```

signature = cle_privée.sign(b"message",
    padding.PKCS1v15(), hashes.SHA256())
cle_publique.verify(signature, b"message",
    padding.PKCS1v15(), hashes.SHA256())

```

— EC (ECDSA / ECDH)

```

from cryptography.hazmat.primitives.asymmetric import ec

```

Génération clé EC

```

cle_privée_ec = ec.generate_private_key(ec.SECP256R1())    P-256
cle_privée_ec = ec.generate_private_key(ec.SECP384R1())    P-384
cle_privée_ec = ec.generate_private_key(ec.SECP521R1())    P-521
cle_privée_ec = ec.generate_private_key(ec.SECP256K1())    secp256k1

```

Signature ECDSA

```

signature = cle_privée_ec.sign(b"message",
    ec.ECDSA(hashes.SHA256()))
cle_publique_ec = cle_privée_ec.public_key()
cle_publique_ec.verify(signature, b"message",
    ec.ECDSA(hashes.SHA256()))

```

ECDH (échange de clé)

```

from cryptography.hazmat.primitives.asymmetric.ec import ECDH
cle_a = ec.generate_private_key(ec.SECP256R1())
cle_b = ec.generate_private_key(ec.SECP256R1())
secret_partagé_a = cle_a.exchange(ECDH(), cle_b.public_key())
secret_partagé_b = cle_b.exchange(ECDH(), cle_a.public_key())
# secret_partagé_a == secret_partagé_b

```

```

ED25519 / ED448 :
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey
from cryptography.hazmat.primitives.asymmetric.ed448 import Ed448PrivateKey

cle = Ed25519PrivateKey.generate()
signature = cle.sign(b"message")
cle.public_key().verify(signature, b"message")

cle448 = Ed448PrivateKey.generate()
signature448 = cle448.sign(b"message")

X25519 (ECDH Curve25519) :
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey

cle_a = X25519PrivateKey.generate()
cle_b = X25519PrivateKey.generate()
secret_a = cle_a.exchange(cle_b.public_key())
secret_b = cle_b.exchange(cle_a.public_key())
# secret_a == secret_b

```

— SÉRIALISATION DES CLÉS —

```

from cryptography.hazmat.primitives import serialization

# Clé privée → PEM
pem = cle_privée.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption(),
)
# Clé privée chiffrée
pem = cle_privée.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.BestAvailableEncryption(b"motdepasse"),
)
# Formats de clé privée
serialization.PrivateFormat.PKCS8                PKCS#8 (standard)
serialization.PrivateFormat.TraditionalOpenSSL    Format OpenSSL traditionnel
serialization.PrivateFormat.Raw                  Bytes bruts (Ed25519, X25519)
serialization.PrivateFormat.OpenSSH               Format OpenSSH

# Clé publique → PEM
pem_pub = cle_publique.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo,
)
# Formats de clé publique
serialization.PublicFormat.SubjectPublicKeyInfo    SPKI standard
serialization.PublicFormat.PKCS1                  PKCS#1 (RSA seulement)
serialization.PublicFormat.Raw                     Bytes bruts (Ed25519, X25519)
serialization.PublicFormat.OpenSSH                 Format OpenSSH

# DER
der = cle_privée.private_bytes(
    encoding=serialization.Encoding.DER,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption(),
)

# Chargement depuis PEM
from cryptography.hazmat.primitives.serialization import load_pem_private_key
cle = load_pem_private_key(pem_bytes, password=None)
cle = load_pem_private_key(pem_bytes, password=b"motdepasse")
cle_pub = serialization.load_pem_public_key(pem_pub_bytes)
cle = serialization.load_der_private_key(der_bytes, password=None)
cle_pub = serialization.load_der_public_key(der_bytes)
cle_ssh = serialization.load_ssh_public_key(ssh_bytes)

```

— CERTIFICATS X.509 —

```

from cryptography import x509
from cryptography.x509.oid import NameOID, ExtendedKeyUsageOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa
import datetime

```

```

# Lire un certificat
from cryptography.x509 import load_pem_x509_certificate
with open("cert.pem", "rb") as f:
    cert = load_pem_x509_certificate(f.read())
cert.subject          Sujet (Name)
cert.issuer           Émetteur (Name)
cert.not_valid_before_utc Début validité (datetime UTC)
cert.not_valid_after_utc Fin validité (datetime UTC)
cert.serial_number     Numéro de série
cert.version           Version X.509
cert.signature         Bytes de la signature
cert.signature_hash_algorithm Algo de signature
cert.public_key()      Clé publique
cert.extensions        Extensions X.509
cert.fingerprint(hashes.SHA256()) Empreinte SHA-256

# Extensions courantes
cert.extensions.get_extension_for_oid(
    x509.ExtensionOID.SUBJECT_ALTERNATIVE_NAME).value.get_values_for_type(x509.DNSName)
cert.extensions.get_extension_for_oid(
    x509.ExtensionOID.BASIC_CONSTRAINTS).value.ca      True si CA
cert.extensions.get_extension_for_oid(
    x509.ExtensionOID.KEY_USAGE).value.digital_signature

# Créer un certificat auto-signé
cle = rsa.generate_private_key(public_exponent=65537, key_size=2048)
sujet = émetteur = x509.Name([
    x509.NameAttribute(NameOID.COUNTRY_NAME, "FR"),
    x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Mon Org"),
    x509.NameAttribute(NameOID.COMMON_NAME, "example.com"),
])
cert = (
    x509.CertificateBuilder()
        .subject_name(sujet)
        .issuer_name(émetteur)
        .public_key(cle.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.datetime.now(datetime.timezone.utc))
        .not_valid_after(datetime.datetime.now(datetime.timezone.utc)
            + datetime.timedelta(days=365))
        .add_extension(
            x509.SubjectAlternativeName([
                x509.DNSName("example.com"),
                x509.DNSName("www.example.com"),
                x509.IPAddress(ipaddress.IPv4Address("192.168.1.1")),
            ]),
            critical=False,
        )
        .add_extension(
            x509.BasicConstraints(ca=False, path_length=None),
            critical=True,
        )
        .sign(cle, hashes.SHA256())
    )
cert_pem = cert.public_bytes(serialization.Encoding.PEM)

```

— DÉRIVATION DE CLÉ (KDF) —

```

from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from cryptography.hazmat.primitives.kdf.hkdf import HKDF, HKDFExpand
from cryptography.hazmat.primitives.kdf.concatkdf import ConcatKDFHash
from cryptography.hazmat.primitives.kdf.x963kdf import X963KDF

# PBKDF2
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,
    salt=sal, iterations=600000)
cle = kdf.derive(b"motdepasse")
kdf2 = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,
    salt=sal, iterations=600000)
kdf2.verify(b"motdepasse", cle)      Lève InvalidKey si incorrect

# Scrypt
kdf = Scrypt(salt=sal, length=32, n=2**14, r=8, p=1)
cle = kdf.derive(b"motdepasse")

```

```

# HKDF (dérivation depuis un secret à haute entropie)
hkdf = HKDF(algorithm=hashes.SHA256(), length=32,
            salt=None, info=b"contexte")
cle = hkdf.derive(secret_ecdh)

# HKDFExpand (si extrait déjà fait)
hkdf_expand = HKDFExpand(algorithm=hashes.SHA256(), length=32,
                        info=b"contexte")
cle = hkdf_expand.derive(prk)

```

— CHIFFREMENT FERNET (haut niveau, symétrique) —

```

from cryptography.fernet import Fernet, MultiFernet

# Génération et chiffrement
cle = Fernet.generate_key()          Clé base64 urlsafe
f = Fernet(cle)
token = f.encrypt(b"message secret") Chiffre (bytes)
clair = f.decrypt(token)             Déchiffre
clair = f.decrypt(token, ttl=60)      Avec TTL en secondes (expire)
f.extract_timestamp(token)           Timestamp du token

# Rotation de clés
cle_nouvelle = Fernet.generate_key()
cle_ancienne = cle
mf = MultiFernet([Fernet(cle_nouvelle), Fernet(cle_ancienne)])
token = mf.encrypt(b"message")       Chiffre avec la nouvelle
clair = mf.decrypt(token_ancien)      Essaie les deux clés
token_rotaté = mf.rotate(token_ancien) Re-chiffre avec nouvelle clé

# Fernet depuis mot de passe
import base64
sel = secrets.token_bytes(16)
kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32,
                 salt=sel, iterations=600000)
cle_fernet = base64.urlsafe_b64encode(kdf.derive(b"motdepasse"))
f = Fernet(cle_fernet)

```

```
[py] Mots de passe – bcrypt, argon2, passlib
```

```

DESCRIPTION : Stockage sécurisé de mots de passe avec algorithmes dédiés
POURQUOI    : Résistance aux attaques GPU/ASIC, sel automatique
CONTEXTE    : Authentification utilisateur, bases de données
INSTALLATION : pip install bcrypt argon2-cffi passlib

```

— BCRYPT —

```

import bcrypt

mdp = b"monmotdepasse"
hash_mdp = bcrypt.hashpw(mdp, bcrypt.gensalt())          Hacher (sel inclus)
hash_mdp = bcrypt.hashpw(mdp, bcrypt.gensalt(rounds=12)) Avec coût (défaut=12)
hash_mdp = bcrypt.hashpw(mdp, bcrypt.gensalt(rounds=14)) Coût 14 (plus lent)
bcrypt.checkpw(mdp, hash_mdp)                            Vérifier → True/False
bcrypt.gensalt()                                           Sel aléatoire seul

```

— ARGON2 —

```

from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError, VerificationError

ph = PasswordHasher()                                     Paramètres recommandés
ph = PasswordHasher(
    time_cost=3,                                          Nb d'itérations
    memory_cost=65536,                                    RAM en KiB (64 MiB)
    parallelism=4,                                         Nb de threads
    hash_len=32,                                           Longueur du hash
    salt_len=16,                                           Longueur du sel
    encoding="utf-8",
)
hash_mdp = ph.hash("motdepasse")                          Hacher
ph.verify(hash_mdp, "motdepasse")                         Vérifier (lève exception si faux)
try:
    ph.verify(hash_mdp, "tentative")
except VerifyMismatchError:

```

```
print("Mot de passe incorrect")
ph.check_needs_rehash(hash_mdp) True si rehash recommandé
```

```
PASSLIB
from passlib.context import CryptContext
from passlib.hash import bcrypt, argon2, pbkdf2_sha256, sha256_crypt

# Contexte multi-schémas
ctx = CryptContext(schemes=["argon2", "bcrypt"],
                  deprecated="auto")
hash_mdp = ctx.hash("motdepasse")
ctx.verify("motdepasse", hash_mdp) True / False
ctx.verify_and_update("motdepasse", hash_mdp) (ok, nouveau_hash_si_mis_à_jour)

# Hash direct
bcrypt.hash("motdepasse", rounds=12)
bcrypt.verify("motdepasse", hash_mdp)
argon2.hash("motdepasse")
argon2.verify("motdepasse", hash_mdp)
pbkdf2_sha256.hash("motdepasse", rounds=600000)
pbkdf2_sha256.verify("motdepasse", hash_mdp)
```

[py] Gestion des secrets – Variables d'environnement & Vaults

DESCRIPTION : Bonnes pratiques de gestion des secrets dans les applications
 POURQUOI : Ne jamais exposer les secrets dans le code source
 CONTEXTE : Applications 12-factor, CI/CD, microservices

VARIABLES D'ENVIRONNEMENT :

```
import os
os.environ.get("SECRET_KEY") None si absente
os.environ.get("SECRET_KEY", "valeur_défaut") Avec valeur par défaut
os.environ["SECRET_KEY"] Lève KeyError si absente
os.environ["DB_PASSWORD"]
os.environ["API_KEY"]
```

```
# Validation à l'initialisation
def charger_secret(nom):
    val = os.environ.get(nom)
    if not val:
        raise EnvironmentError(f"Variable d'environnement '{nom}' manquante")
    return val
```

PYTHON-DOTENV :

```
# pip install python-dotenv
from dotenv import load_dotenv
load_dotenv() Charge .env depuis le répertoire courant
load_dotenv(".env.production") Fichier personnalisé
load_dotenv(override=True) Écrase les variables existantes
load_dotenv(verbose=True) Mode verbeux
from dotenv import dotenv_values
config = dotenv_values(".env") Dict sans modifier os.environ
```

```
# Format .env
# SECRET_KEY=ma_cle_secrete
# DATABASE_URL=postgresql://user:pass@host/db
# API_KEY=abc123
# Ne JAMAIS committer .env dans git (ajouter au .gitignore)
```

PYDANTIC SETTINGS :

```
# pip install pydantic-settings
from pydantic_settings import BaseSettings
from pydantic import SecretStr
```

```
class Paramètres(BaseSettings):
    secret_key: SecretStr
    db_password: SecretStr
    api_key: str
    debug: bool = False

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"
```

```

config = Paramètres()
config.secret_key.get_secret_value()
str(config.secret_key)

```

Valeur réelle
 "*****" (masquée)

```

HASHICORP VAULT :
# pip install hvac
import hvac
client = hvac.Client(url="https://vault:8200",
                    token=os.environ["VAULT_TOKEN"])
client.is_authenticated()
# Lire un secret KV v2
secret = client.secrets.kv.v2.read_secret_version(
    path="mon-app/config",
    mount_point="secret",
)
mot_de_passe = secret["data"]["data"]["db_password"]
# Écrire un secret
client.secrets.kv.v2.create_or_update_secret(
    path="mon-app/config",
    secret={"db_password": "nouveau_mdp"},
)
# Token AppRole
client.auth.approle.login(role_id=role_id, secret_id=secret_id)

```

True si connecté

```

AWS SECRETS MANAGER :
# pip install boto3
import boto3, json
sm = boto3.client("secretsmanager", region_name="eu-west-1")
response = sm.get_secret_value(SecretId="mon-app/production/db")
secret = json.loads(response["SecretString"])
mot_de_passe = secret["password"]

```

```

AZURE KEY VAULT :
# pip install azure-keyvault-secrets azure-identity
from azure.keyvault.secrets import SecretClient
from azure.identity import DefaultAzureCredential
client = SecretClient(vault_url="https://monvault.vault.azure.net/",
                    credential=DefaultAzureCredential())
secret = client.get_secret("nom-du-secret")
valeur = secret.value

```

```

GCP SECRET MANAGER :
# pip install google-cloud-secret-manager
from google.cloud import secretmanager
client = secretmanager.SecretManagerServiceClient()
nom = "projects/mon-projet/secrets/mon-secret/versions/latest"
response = client.access_secret_version(request={"name": nom})
valeur = response.payload.data.decode("utf-8")

```

[py] JWT – Tokens d'authentification

DESCRIPTION : Crée et vérifie des JSON Web Tokens signés
 POURQUOI : Authentification sans état, échange sécurisé d'informations
 CONTEXTE : APIs REST, microservices, authentification SSO
 INSTALLATION : pip install pyjwt cryptography

```

PYJWT :
import jwt, datetime

# Clé symétrique (HMAC)
SECRET = secrets.token_bytes(32)
payload = {"sub": "user123", "exp": datetime.datetime.utcnow()
          + datetime.timedelta(hours=1)}
token = jwt.encode(payload, SECRET, algorithm="HS256")
données = jwt.decode(token, SECRET, algorithms=["HS256"])

# Clé asymétrique (RSA)
from cryptography.hazmat.primitives.asymmetric import rsa
cle_privée = rsa.generate_private_key(65537, 2048)
token = jwt.encode(payload, cle_privée, algorithm="RS256")
données = jwt.decode(token, cle_privée.public_key(), algorithms=["RS256"])

```

```

# EC
from cryptography.hazmat.primitives.asymmetric import ec
cle_ec = ec.generate_private_key(ec.SECP256R1())
token = jwt.encode(payload, cle_ec, algorithm="ES256")
données = jwt.decode(token, cle_ec.public_key(), algorithms=["ES256"])

# Options
jwt.decode(token, SECRET, algorithms=["HS256"],
            options={"verify_exp": True,          Vérifier expiration
                    "verify_iat": True,         Vérifier issued_at
                    "require": ["exp", "sub"]})  Champs requis

# Exceptions
jwt.ExpiredSignatureError      Token expiré
jwt.InvalidTokenError         Token invalide
jwt.DecodeError               Erreur de décodage
jwt.InvalidSignatureError     Signature invalide
jwt.MissingRequiredClaimError Champ requis manquant

CLAIMS STANDARD :
"iss" : "émetteur"           Issuer
"sub" : "sujet"              Subject
"aud" : "audience"          Audience
"exp" : datetime.utcnow() + timedelta(hours=1) Expiration (timestamp UNIX)
"nbf" : datetime.utcnow()    Not Before
"iat" : datetime.utcnow()    Issued At
"jti" : secrets.token_hex(16) JWT ID (unicité)

ALGORITHMES :
"HS256", "HS384", "HS512"    HMAC-SHA (symétrique)
"RS256", "RS384", "RS512"    RSA-SHA (asymétrique)
"ES256", "ES384", "ES512"    ECDSA-SHA (asymétrique)
"PS256", "PS384", "PS512"    RSA-PSS (asymétrique)
"EdDSA"                      Ed25519/Ed448

```

```
[py] TLS avec httpx / requests
```

```

DESCRIPTION : Requêtes HTTPS avec contrôle TLS fin
POURQUOI    : APIs sécurisées, authentification mutuelle (mTLS)
CONTEXTE     : Clients HTTP, microservices, intégrations

REQUESTS :
import requests

requests.get("https://example.com")           HTTPS standard
requests.get("https://example.com", verify=True) Vérification CA (défaut)
requests.get("https://example.com", verify="/chemin/ca.pem") CA custom
requests.get("https://example.com", verify=False) Désactiver (dev uniquement)
requests.get("https://example.com",
            cert=("cert.pem", "cle.pem"))      mTLS (cert client)
requests.get("https://example.com",
            cert=("cert.pem", "cle.pem"),
            verify="ca.pem")                   mTLS + CA custom
session = requests.Session()
session.verify = "ca.pem"
session.cert = ("cert.pem", "cle.pem")
session.get("https://example.com")

HTTPX :
import httpx, ssl

ctx = ssl.create_default_context(cafile="ca.pem")
client = httpx.Client(verify=ctx)
client = httpx.Client(verify="ca.pem")
client = httpx.Client(verify=False)           Désactiver (dev)
client = httpx.Client(cert=("cert.pem", "cle.pem")) mTLS
client = httpx.Client(cert=("cert.pem", "cle.pem"),
                    verify="ca.pem")         mTLS + CA custom
with httpx.Client(verify="ca.pem") as client:
    r = client.get("https://example.com")

# Async
async with httpx.AsyncClient(verify="ca.pem") as client:

```



```
r = await client.get("https://example.com")
```

RÉCAPITULATIF – CHOISIR LE BON OUTIL

BESOIN	OUTIL RECOMMANDÉ
Générer un token / nonce / secret	<code>secrets.token_urlsafe()</code>
Comparer deux tokens en sécurité	<code>secrets.compare_digest()</code>
Mot de passe fort	<code>secrets.choice()</code> + <code>secrets.token_bytes()</code>
Hacher un mot de passe	<code>argon2-cffi</code> ou <code>bcrypt</code> (PAS <code>hashlib</code>)
Vérifier un mot de passe	<code>ph.verify()</code> ou <code>bcrypt.checkpw()</code>
Intégrité d'un fichier	<code>hashlib.sha256()</code> ou <code>sha3_256()</code>
HMAC d'un message	<code>hmac.new()</code> ou <code>hashlib.blake2b(key=...)</code>
Dériver une clé depuis un mot de passe	<code>hashlib.scrypt()</code> ou <code>PBKDF2HMAC</code>
Dériver une clé depuis un secret fort	<code>HKDF</code> (cryptography)
Chiffrement symétrique rapide	<code>AESGCM</code> ou <code>ChaCha20Poly1305</code> (cryptography)
Chiffrement symétrique simple (haut niveau)	<code>Fernet</code> (cryptography)
Chiffrement asymétrique / signature	<code>RSA</code> (OAEP/PSS) ou <code>EC</code> (ECDSA)
Signature légère / moderne	<code>Ed25519</code> (cryptography)
Échange de clé	<code>X25519</code> ou <code>ECDH P-256</code>
Auth sans état (API)	<code>JWT</code> (PyJWT)
Certificats TLS auto-signés	<code>openssl req -x509</code> ou <code>x509.CertificateBuilder</code>
Connexion TLS cliente	<code>ssl.create_default_context()</code>
mTLS	<code>ssl.load_cert_chain()</code> + <code>requests/httpx cert=</code>
Inspecter un certificat	<code>openssl x509 -text</code> ou <code>x509.load_pem_x509_certificate</code>
Stocker les secrets (app)	Variables d'env + <code>python-dotenv</code>
Stocker les secrets (prod)	<code>Vault</code> / <code>AWS Secrets Manager</code> / <code>Azure Key Vault</code>

— RÈGLES D'OR —

- ✗ Ne jamais utiliser `random` pour la sécurité → utiliser `secrets` ou `os.urandom()`
- ✗ Ne jamais comparer des tokens avec `==` → utiliser `secrets.compare_digest()`
- ✗ Ne jamais stocker les mots de passe en clair ni en SHA → `bcrypt` ou `argon2`
- ✗ Ne jamais désactiver `verify=False` en production
- ✗ Ne jamais committer de secrets dans `git`
- ✗ Ne jamais utiliser MD5 ou SHA-1 pour la sécurité
- ✗ Ne jamais utiliser AES-CBC sans MAC séparé → préférer AES-GCM (AEAD)
- ✗ Ne jamais réutiliser un nonce/IV avec la même clé (surtout GCM)
- ✓ Utiliser `-pbkdf2` avec `openssl enc` (pas l'ancien `EVP_BytesToKey`)
- ✓ Inclure un TTL dans les tokens (JWT exp, Fernet ttl)
- ✓ Utiliser TLS 1.2 minimum, TLS 1.3 recommandé
- ✓ Utiliser des certificats avec SAN (pas juste CN)
- ✓ Renouveler les certificats avant expiration (Let's Encrypt : 60 jours)
- ✓ Utiliser des clés d'au moins 2048 bits (RSA) ou P-256 (EC)

— TAILLES DE CLÉS RECOMMANDÉES (2024) —

ALGORITHME	TAILLE MINIMALE	RECOMMANDÉE	REMARQUE
AES symétrique	128 bits	256 bits	GCM ou CTR
RSA	2048 bits	4096 bits	OAEP pour chiffrement, PSS pour signature
EC (ECDSA/ECDH)	256 bits (P-256)	384 bits (P-384)	secp256k1 pour crypto-monnaies
Ed25519	256 bits (fixe)	256 bits	Recommandé pour nouvelles applis
X25519	256 bits (fixe)	256 bits	Remplace ECDH pour DH
DH classique	2048 bits	3072 bits	Préférer ECDH
Hash (intégrité)	SHA-256	SHA-3-256	Ne pas utiliser MD5/SHA-1
Hash (MDP)	<code>bcrypt cost=12</code>	<code>argon2id</code>	Jamais SHA-256 direct pour MDP

— ENTROPIE DES TOKENS —

FONCTION	SORTIE	ENTROPIE
<code>secrets.token_bytes(16)</code>	16 octets	128 bits
<code>secrets.token_bytes(32)</code>	32 octets	256 bits
<code>secrets.token_hex(16)</code>	32 chars hex	128 bits
<code>secrets.token_hex(32)</code>	64 chars hex	256 bits
<code>secrets.token_urlsafe(16)</code>	~22 chars	128 bits
<code>secrets.token_urlsafe(32)</code>	~43 chars	256 bits
<code>openssl rand -hex 32</code>	64 chars hex	256 bits
<code>openssl rand -base64 32</code>	~44 chars	256 bits